Report of cloud computing

5/6/2025

Prepared by:
Azzman Nor Elimane
El ghazi Silia

Supervised by:
Pr. Sara Ben Ahrach

Table of Contents

Introduction

In today's rapidly evolving digital landscape, cloud computing has emerged as a foundational pillar for modern IT infrastructures. Enterprises, institutions, and governments are increasingly adopting cloud-based solutions to enhance scalability, reduce operational costs, and accelerate service delivery. This term project, titled "Deploying and Managing a Cloud Environment Using OpenStack," was undertaken as part of the Introduction to Cloud Computing course at the National School of Applied Sciences of Tangier. The primary goal is to simulate a real-world cloud deployment scenario using OpenStack, a powerful open-source cloud platform widely adopted in the industry.

The project is structured to offer both technical depth and educational value, providing students with hands-on experience in installing, configuring, and managing a private cloud environment. From understanding the core principles of cloud service models (IaaS, PaaS, SaaS) to the practical deployment of virtual machines, storage, and networking services, the project bridges the gap between theory and real-world application. It also emphasizes critical skills such as problem-solving, debugging, system integration, and documentation — all essential for future cloud engineers and system architects.

By focusing on OpenStack, this project not only introduces students to one of the most robust and flexible cloud orchestration platforms but also illustrates the strategic significance of mastering infrastructure-as-a-service (IaaS) concepts in an era where digital transformation is reshaping all sectors. The practical implementation of services like Nova, Neutron, Cinder, and Horizon demonstrates how organizations build and scale private cloud environments to meet dynamic business needs. This exposure equips students with a foundational competency that is increasingly demanded in careers related to DevOps, system administration, and cloud architecture.

Fundamentals of Cloud Computing

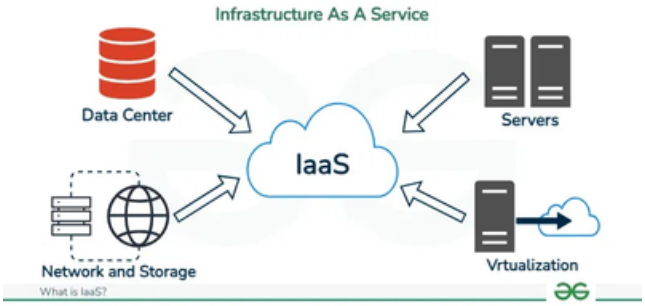
Cloud computing refers to the delivery of computing services—including servers, storage, databases, networking, software, and analytics—over the internet ("the cloud") to offer faster innovation, flexible resources, and economies of scale. Instead of owning and maintaining physical data centers and servers, organizations can access technology services on-demand from cloud providers or deploy their own private cloud using tools such as OpenStack. Cloud computing is generally classified into three primary service models: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS).

DEFINITIONS OF SERVICE MODELS

Infrastructure as a Service (IaaS):

IaaS provides virtualized computing resources over the internet. It includes fundamental services such as virtual machines, networks, and storage. Users manage the operating systems, applications, and runtime environments, while the provider manages the infrastructure.

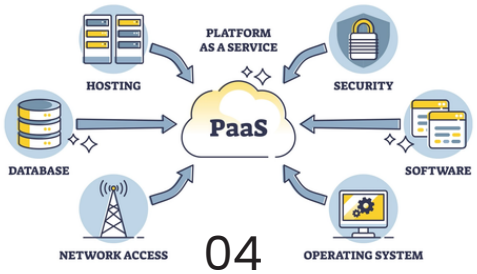
Example: A startup launching a web application might use OpenStack or Amazon EC2 to provision virtual machines and configure storage and network settings without purchasing physical hardware.



Platform as a Service (PaaS):

PaaS delivers a platform that allows developers to build, deploy, and manage applications without managing the underlying infrastructure. It abstracts server and runtime management, enabling faster development cycles.

Example: Developers can use Google App Engine or Heroku to deploy and scale applications written in Python, Node.js, or Java without worrying about server setup or OS maintenance.



Fundamentals of Cloud Computing

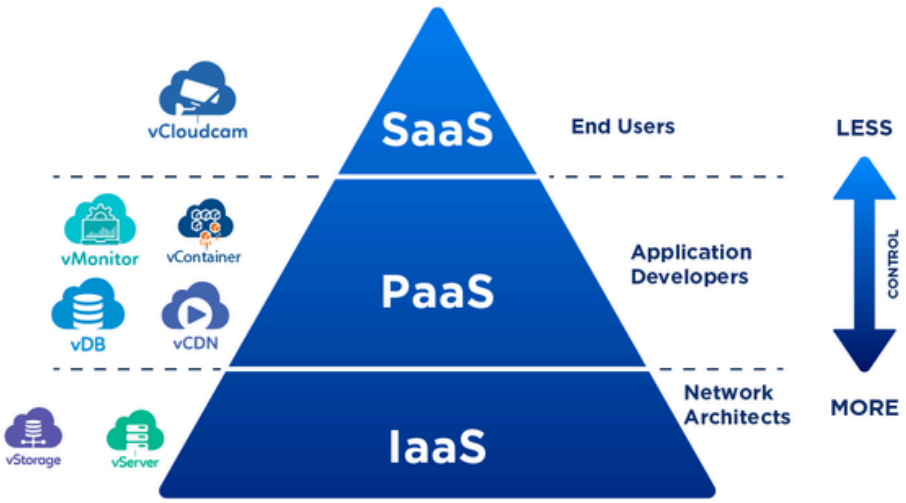
DEFINITIONS OF SERVICE MODELS

Software as a Service (SaaS):

SaaS offers fully functional, user-ready applications over the internet. Users access the software through a web browser or API, while the provider handles everything from infrastructure to updates.

Example: Tools like Google Workspace (Gmail, Docs, Drive) or Salesforce CRM allow businesses to use robust productivity and customer management tools without any installation or maintenance burden.

SUMMARY

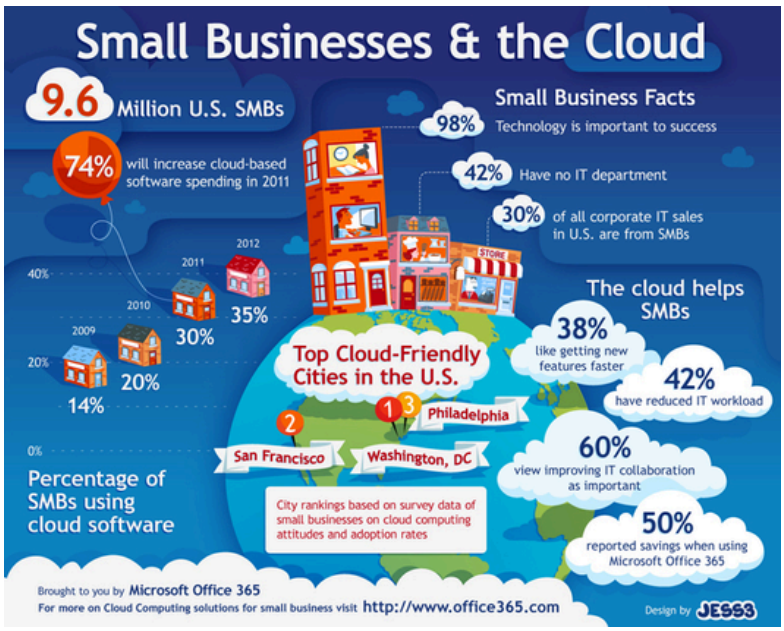


Fundamentals of Cloud Computing

COMPARISON AND REAL-WORLD USE CASES

Feature	IaaS	PaaS	SaaS
User Control	High (OS, apps, middleware)	Medium (apps only)	Low (just configuration)
Use Case	System administrators, DevOp	Developers, data scientists	End users, business professionals
Example Providers	OpenStack, AWS EC2, Microsof	Google App Engine, Heroku	Gmail, Dropbox, Zoom

- A university IT department might use OpenStack (IaaS) to create a private cloud for hosting internal student and faculty services.
- A software development team could choose PaaS to build and test a mobile app without worrying about the server environment.
- A small business may rely on SaaS for day-to-day operations using platforms like Microsoft 365 for communication and file sharing.



Fundamentals of Cloud Computing

BENEFITS AND CHALLENGES OF CLOUD COMPUTING

Benefits:

- Scalability: Cloud resources can be scaled up or down based on demand, making it ideal for dynamic workloads such as seasonal web traffic.
- Cost-efficiency: Pay-as-you-go pricing models reduce capital expenditure, especially for startups and academic institutions.
- Accessibility: Services can be accessed globally, enhancing collaboration and productivity.
- Speed of Deployment: Infrastructure and platforms can be provisioned in minutes, speeding up innovation and time-to-market.

Challenges:

- Security and Privacy: Sensitive data stored on the cloud requires robust encryption and access control mechanisms, especially in regulated sectors like healthcare or education.
- Downtime and Dependence: If the cloud provider experiences outages, service delivery is disrupted, affecting business continuity.
- Vendor Lock-in: Some cloud providers use proprietary technologies that can make migration to another platform difficult.
- Skill Gap: Organizations often face a lack of trained personnel to manage cloud environments effectively.

OpenStack Architecture

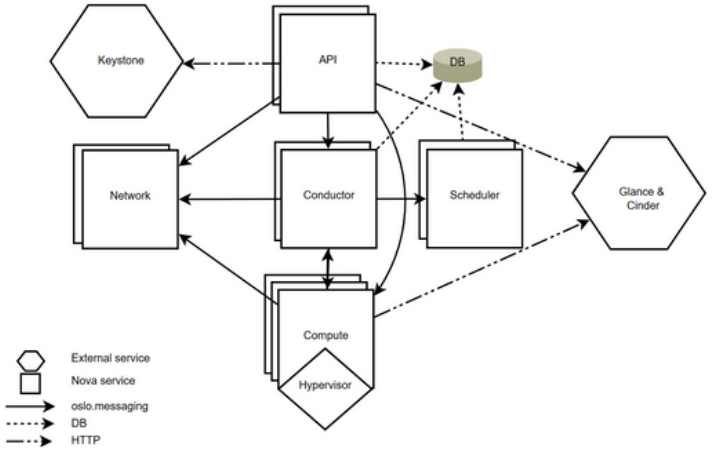
OpenStack is a modular cloud computing platform designed to control large pools of compute, storage, and networking resources throughout a datacenter. Its architecture is composed of several interrelated components, each serving a specific function within the cloud environment. Below is an overview of the primary components:

NOVA (COMPUTE)

Nova is the compute service in OpenStack responsible for provisioning and managing large networks of virtual machines. It acts as a controller, orchestrating the lifecycle of compute instances, including creation, scheduling, and termination. Nova supports various hypervisors like KVM, Xen, and VMware, providing flexibility in deployment

Example: When a user requests a new virtual machine, Nova schedules the request to an appropriate compute node, interacts with the hypervisor to launch the instance, and manages its state throughout its lifecycle.

Visual Representation:

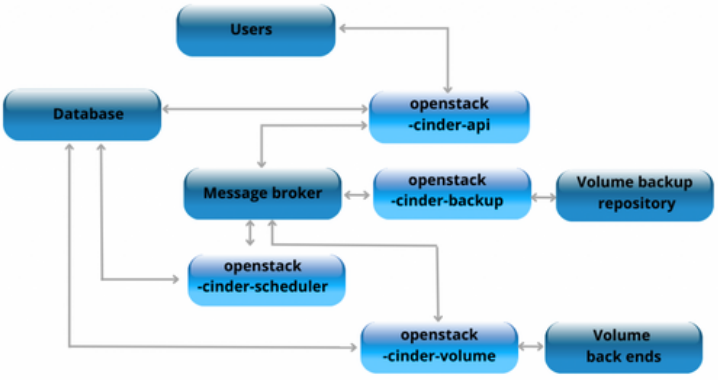


CINDER (BLOCK STORAGE)

Cinder provides block storage services to OpenStack. It allows users to create and manage persistent volumes that can be attached to virtual machines. These volumes function similarly to physical hard drives and can be used for storing data that needs to persist beyond the life of a single VM.

Example: A database application running on a VM can use a Cinder volume to store its data, ensuring that the information remains intact even if the VM is terminated or restarted.

Visual Representation:



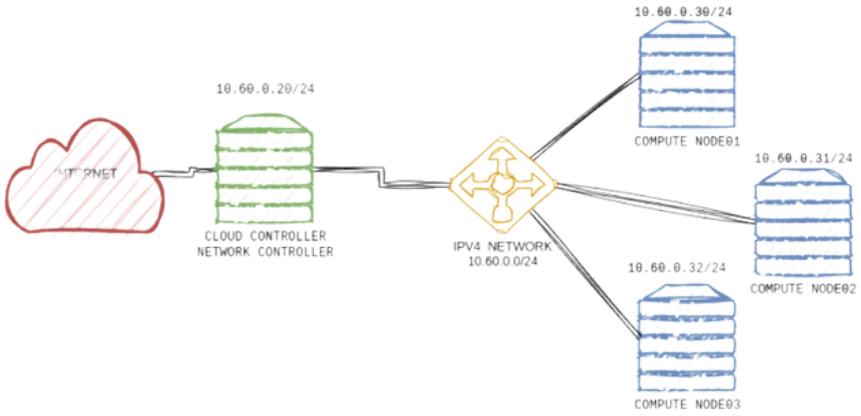
OpenStack Architecture

SWIFT (OBJECT STORAGE)

Swift is the object storage system within OpenStack, designed to store and retrieve unstructured data at scale. It stores data as objects within containers, providing a scalable and redundant storage solution ideal for backups, archives, and media files.

Example: A company can use Swift to store large amounts of image or video files, which can be accessed and managed via RESTful APIs

Visual Representation:

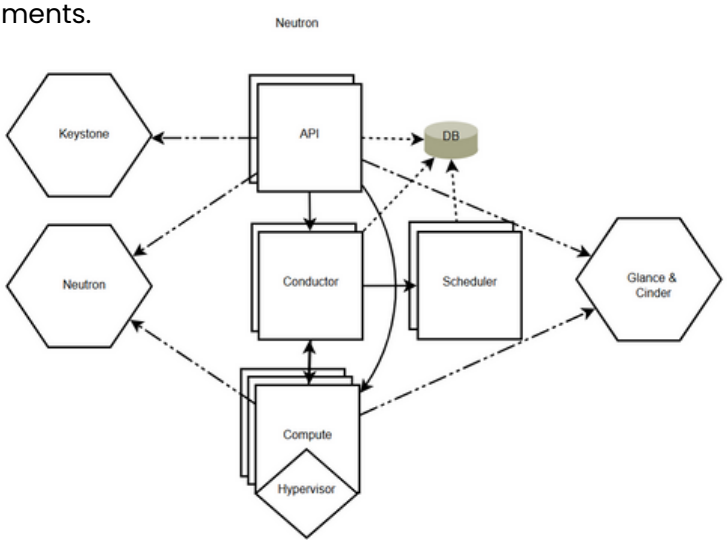


NEUTRON (NETWORKING)

Neutron is the networking component of OpenStack, offering "networking as a service" between interface devices managed by other OpenStack services. It enables users to create and manage networks, subnets, and routers, and supports advanced services like firewalls and load balancers.

Example: An administrator can define a virtual network topology, including subnets and routers, to isolate and manage traffic between different tenant environments.

Visual Representation:



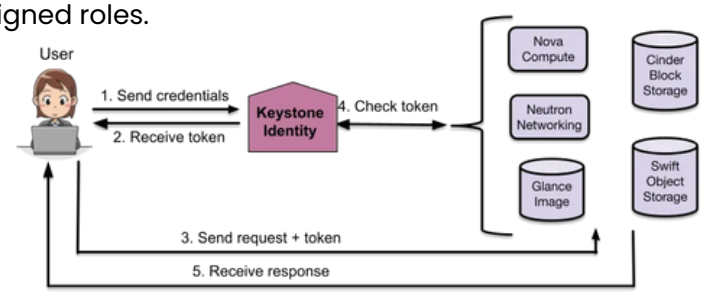
OpenStack Architecture

KEYSTONE (IDENTITY MANAGEMENT)

Keystone is the identity service used for authentication and high-level authorization. It provides a central directory of users mapped to the OpenStack services they can access. Keystone supports multiple forms of authentication and is essential for securing the OpenStack environment.

Example: When a user attempts to access the OpenStack dashboard, Keystone verifies their credentials and determines their permissions based on assigned roles.

Visual Representation:

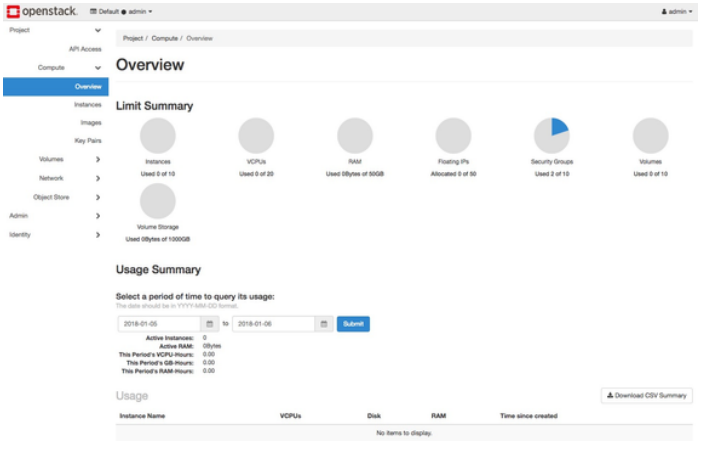


HORIZON (DASHBOARD)

Horizon is the web-based dashboard for OpenStack, providing a graphical interface for users and administrators to access, provision, and automate cloud-based resources. It interacts with the underlying OpenStack services through APIs, allowing for the management of instances, volumes, networks, and more.

Example: A user can log into Horizon to launch a new virtual machine, attach a volume, and configure networking—all through an intuitive web interface.

Visual Representation:



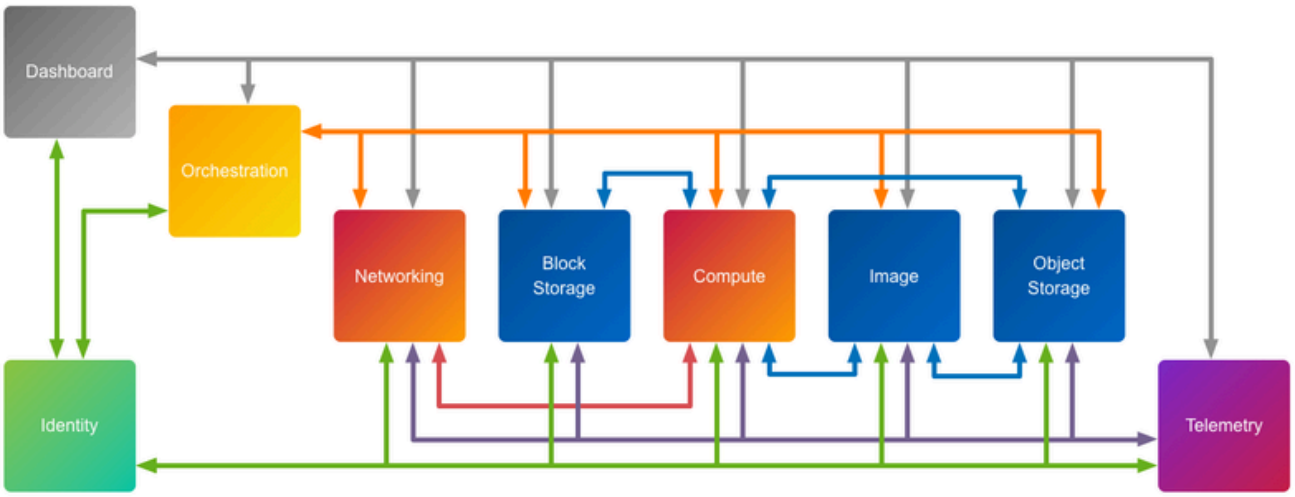
OpenStack Architecture

COMPONENT INTERACTION

The OpenStack components interact seamlessly to provide a cohesive cloud computing platform:

- User Authentication: Users authenticate through Keystone, which provides tokens for accessing other services.
- Resource Provisioning: Using Horizon or CLI tools, users request resources (e.g., VMs via Nova).
- Networking: Neutron sets up the necessary networking for the instances.
- Storage: Cinder provides block storage volumes, while Swift handles object storage needs.
- Monitoring and Management: Horizon allows users to monitor and manage their resources through a unified dashboard.

This modular architecture ensures scalability, flexibility, and ease of integration, making OpenStack a robust solution for private and public cloud deployments.



OpenStack Installation and Configuration

TOOL CHOICE: DEVSTACK VS. PACKSTACK

OpenStack provides multiple methods for deploying a cloud environment depending on the use case—testing, development, or production. Among the most used deployment tools are DevStack and Packstack, each tailored to specific objectives and system environments.

Feature	DevStack	Packstack
Purpose	Primarily for **development** and testing of OpenStack features	Designed for **simplified deployment** of OpenStack (small-scale prod)
Installation Scope	**Single-node** installations (local or VM)	Supports **multi-node** installations via SSH
Platform Support	Optimized for **Ubuntu/Debian** systems	Best suited for **CentOS / RHEL** distributions
Tooling Approach	Bash scripts and environment variables	Uses **Puppet** modules
Speed of Setup	Fast setup, but prone to breaking in newer branches	Slower, but more stable for base configurations
Flexibility	Highly flexible; easy to modify for testing purposes	Less flexible; focuses on standardized deployment
Persistence	**Non-persistent** after reboot (unless manually configured)	**Persistent** services post-reboot
Customization Level	High – ideal for contributors and those testing new features	Medium – suited for sysadmins and new users
Maintenance & Community	Maintained by OpenStack contributors; changes often	Officially supported but **Packstack is deprecated** post-Wallaby release
Use Case Fit	Best for **developers**, students, POCs	Best for **IT admins**, labs, and small pilots
Complexity	Requires understanding of CLI tools and script logs	Easier via guided CLI wizard (`packstack --allinone`)
Typical Command	`./stack.sh` from DevStack repo	`packstack --allinone` or custom answer file

When setting up OpenStack for development or testing purposes, two popular tools are commonly used:

- DevStack: A set of extensible scripts designed for developers to quickly deploy an OpenStack environment. It's ideal for single-node installations and is primarily used on Ubuntu systems.Packstack: A utility that uses Puppet modules to deploy OpenStack on multiple pre-installed servers over SSH automatically. It's suitable for CentOS and RHEL distributions.

OpenStack Installation and Configuration

CONCLUSION: WHY WE CHOSE DEVSTACK

For the purpose of this university project—testing, learning, and deploying a small OpenStack lab on a single VM—DevStack is the ideal choice. It allows quick setup, full access to OpenStack services, and is tailored for education and experimentation. Though not suited for production, its ease of use and flexibility make it the go-to tool for developers and students.

DevStack

SYSTEM REQUIREMENTS AND ENVIRONMENT SETUP

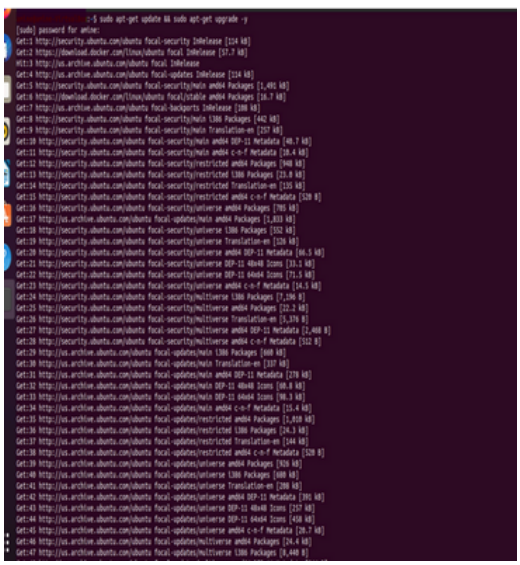
Before installing DevStack, ensure your system meets the following requirements:

- Operating System: Ubuntu 18.04 or later
- Hardware:
  - Minimum 4 GB RAM (8 GB recommended)
  - At least 2 vCPUs
  - 20 GB of free disk space
- Network: Stable internet connection
- User: A non-root user with sudo privileges

STEPS TO PREPARE THE ENVIRONMENT:

1. **Update and Upgrade the System:**

```
sudo apt update && sudo apt upgrade -y
```

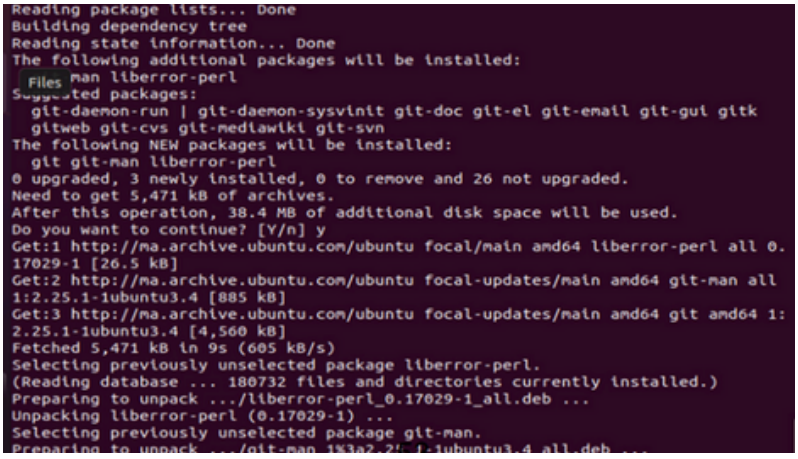


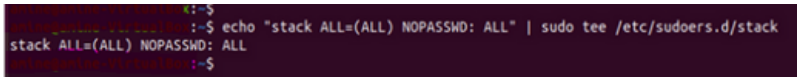
OpenStack Installation and Configuration

STEPS TO PREPARE THE ENVIRONMENT:

2.Install Git and Clone DevStack Repository:

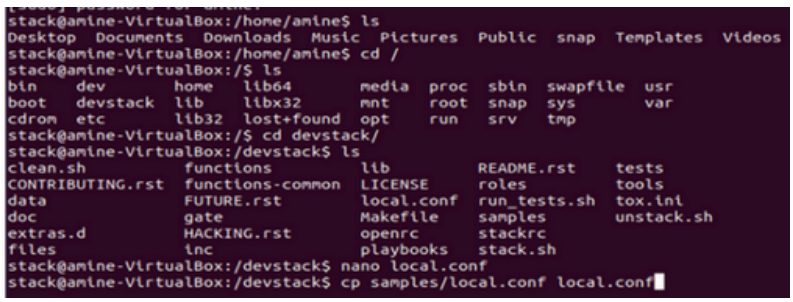
```
sudo apt install git -y
git clone https://opendev.org/openstack/devstack
cd devstack
```

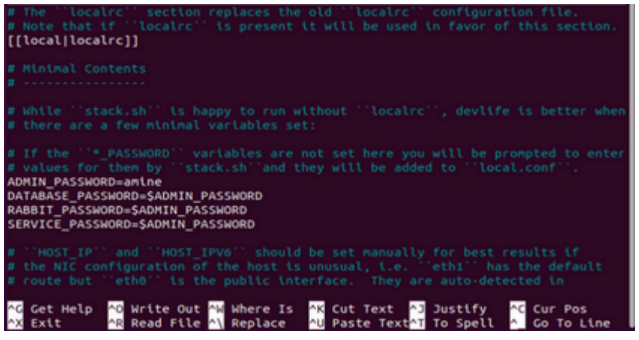




3.Create a Local Configuration File:

```
nano local.conf
```





OpenStack Installation and Configuration

STEPS TO PREPARE THE ENVIRONMENT:

3.Create a Local Configuration File:

```
./stack.sh
```

4.Issues Encountered and Troubleshooting

During the installation of OpenStack with DevStack, we faced several common issues:

- Missing Dependencies: Some required packages were initially missing. We resolved this using:

sudo apt install -f

- Network Configuration Errors: The HOST_IP in local.conf was incorrectly set, causing network-related failures. We used ip a to find and assign the correct IP address.
- Service Failures: Certain services failed to launch. We inspected the logs in /opt/stack/logs/ to identify and fix the issues.
- Reinstallation Needed: In case of critical failures, we cleaned the environment and restarted the setup using:

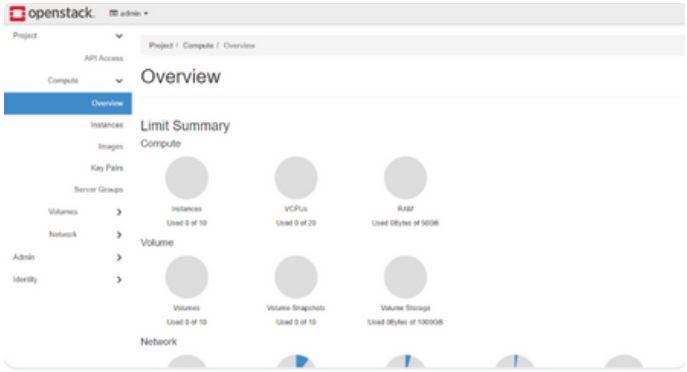
./unstack.sh

./clean.sh

./stack.sh

These steps helped us complete the setup successfully after initial setbacks.

5. Successful Login to the Horizon Dashboard



Virtual Machine Deployment and Management

CREATING VMS WITH NOVA

To deploy virtual machines (VMs) in OpenStack, we utilized the Nova compute service. The process began by selecting an appropriate image (such as Ubuntu 20.04), a flavor defining the compute resources (e.g., m1.small), and a network to connect the instance. We also ensured that a key pair was available for SSH access. Using the OpenStack command-line interface (CLI), the VM was created with the following command:

This command initiates the creation of a VM named web-server with the specified configurations. The instance's status was monitored using openstack server list, and upon reaching the "ACTIVE" state, it was ready for further configuration.

```
openstack server create \
  --image ubuntu-20.04 \
  --flavor m1.small \
  --network private-net \
  --key-name mykey \
  --security-group default \
  web-server
```

DEPLOYING A WEB SERVER (APACHE OR NGINX) ON A VM

After the VM was active, we proceeded to install a web server to serve HTTP requests. Accessing the VM via SSH was achieved using the floating IP assigned to the instance:

```
ssh -i mykey.pem ubuntu@<FLOATING_IP>
```

Once connected, we updated the package lists and installed Apache or Nginx:

For Apache:

```
bash

sudo apt update
sudo apt install apache2 -y
```

For Nginx:

```
bash

sudo apt update
sudo apt install nginx -y
```

Post-installation, the web server's status was verified using systemctl status apache2 or systemctl status nginx. To confirm successful deployment, we navigated to http://<FLOATING_IP> in a web browser, which displayed the default web page.

ASSIGNING A FLOATING IP FOR EXTERNAL ACCESS

To make the VM accessible from external networks, we assigned a floating IP. Floating IPs in OpenStack are public IP addresses that can be dynamically associated with instances. The process involved two main steps:

1. Allocate a Floating IP:

```
openstack floating ip create public
```

2.Associate the Floating IP with the VM:

```
openstack server add floating ip web-server <FLOATING_IP>
```

Virtual Machine Deployment and Management

CONFIGURING SECURITY GROUPS AND FIREWALL RULES

Security groups in OpenStack function as virtual firewalls, controlling inbound and outbound traffic to instances. By default, new instances have restrictive security group rules, so we configured them to allow necessary traffic:

- Create a Security Group:

openstack security group create web-access --description "Allow web traffic"

1. Add Rules to the Security Group:
   - Allow SSH (port 22):

openstack security group rule create --protocol tcp --dst-port 22 web-access

   - Allow HTTP (port 80):

openstack security group rule create --protocol tcp --dst-port 80 web-access

   - Allow HTTPS (port 443):

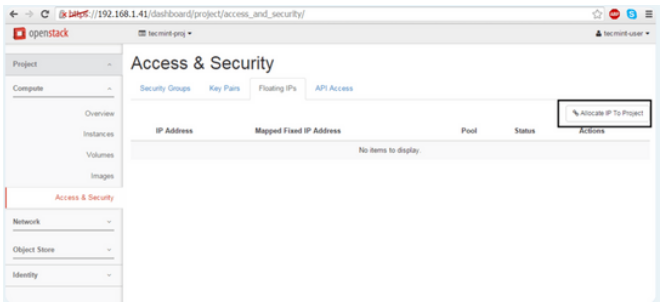
openstack security group rule create --protocol tcp --dst-port 443 web-access

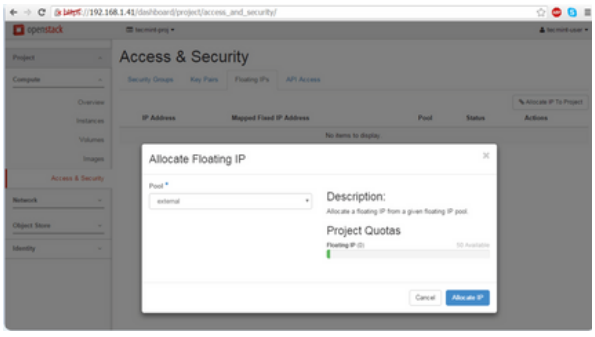
1. Assign the Security Group to the VM:
2. openstack server add security group web-server web-access

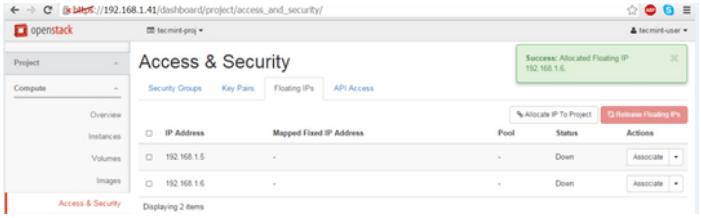
These configurations ensured that the VM could be accessed via SSH and serve web traffic over HTTP and HTTPS. For more information on managing security groups, refer to the OpenStack documentation .

ALL THE STEPS (SCREENSHOTS)

Step 1: Allocate Floating IP to OpenStack



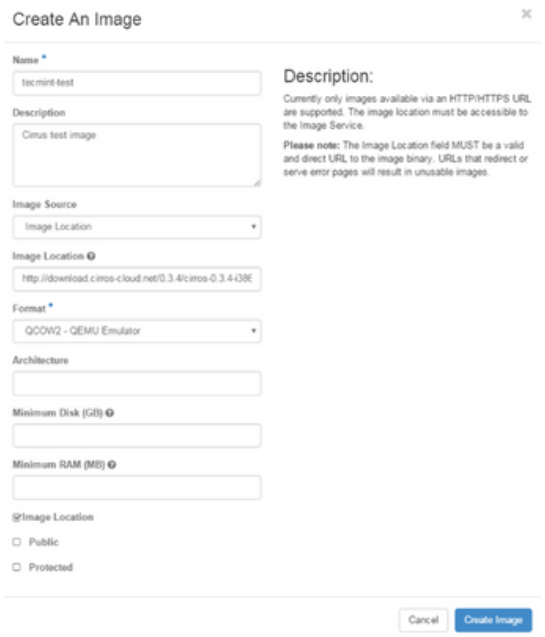


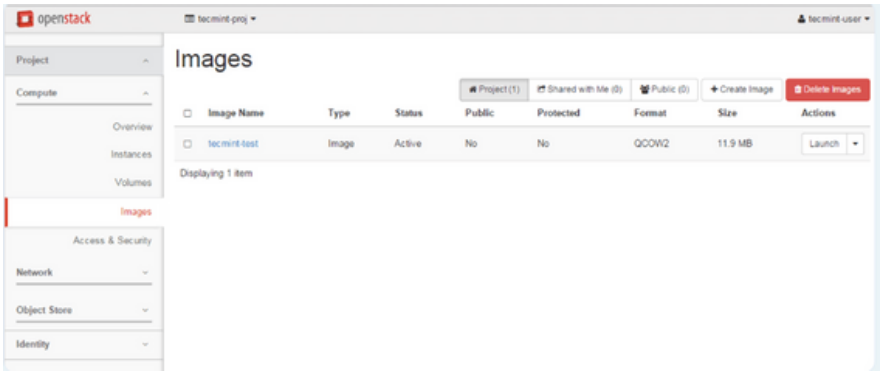


Virtual Machine Deployment and Management

ALL THE STEPS (SCREENSHOTS)

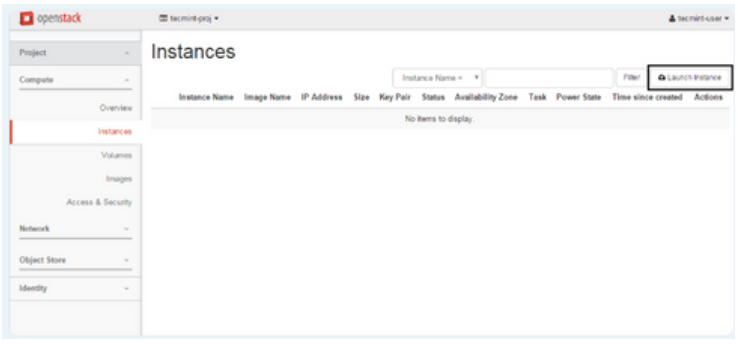
Step 2: Create an OpenStack Image



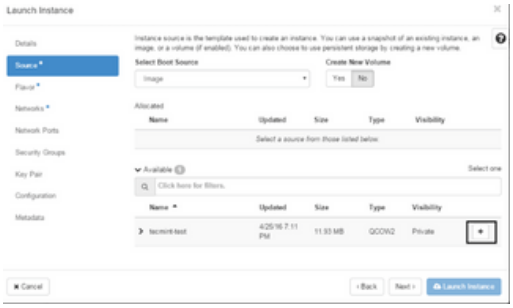


Step 3: Launch an Image Instance in OpenStack

Move to Project -> Instances and hit on Launch Instance button and a new window will appear.



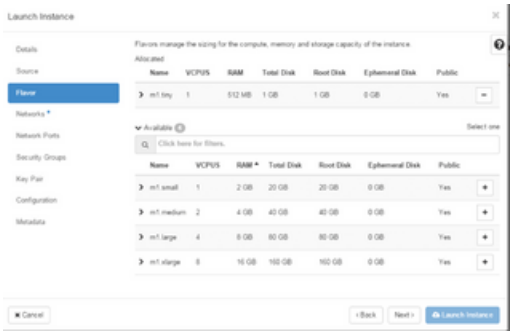
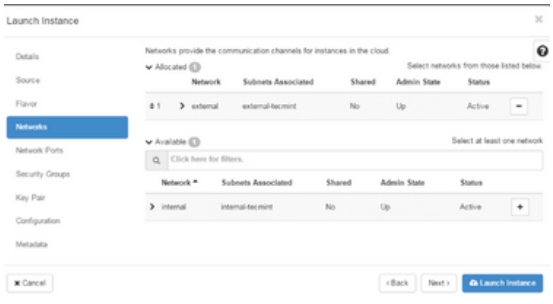
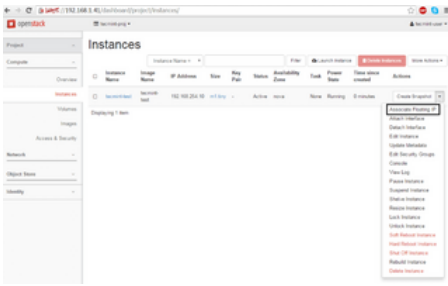
On the first screen add a name for your instance, leave the Availability Zone to nova, use one instance count and hit on Next button to continue.Choose a descriptive Instance Name for your instance because this name will be used to form the virtual machine hostname.



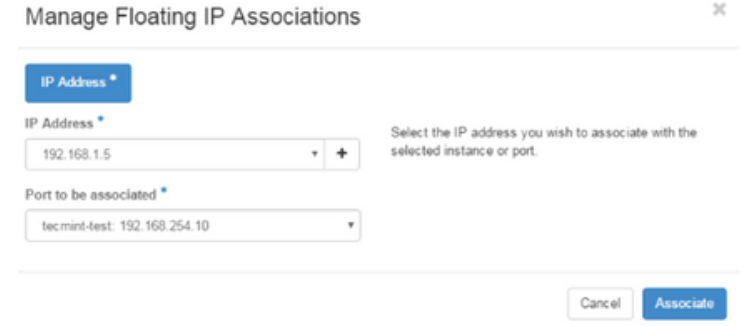
Virtual Machine Deployment and Management

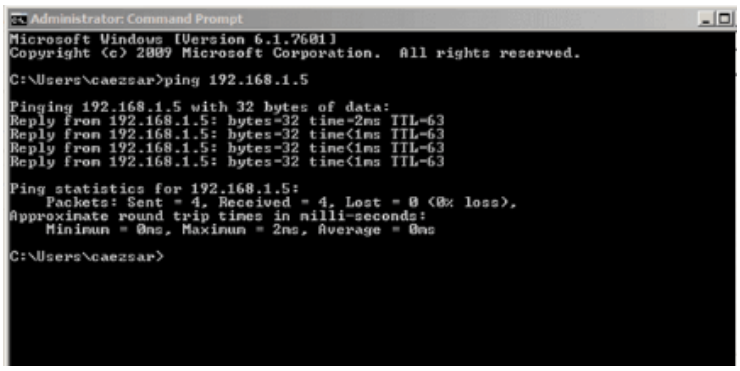
ALL THE STEPS (SCREENSHOTS)

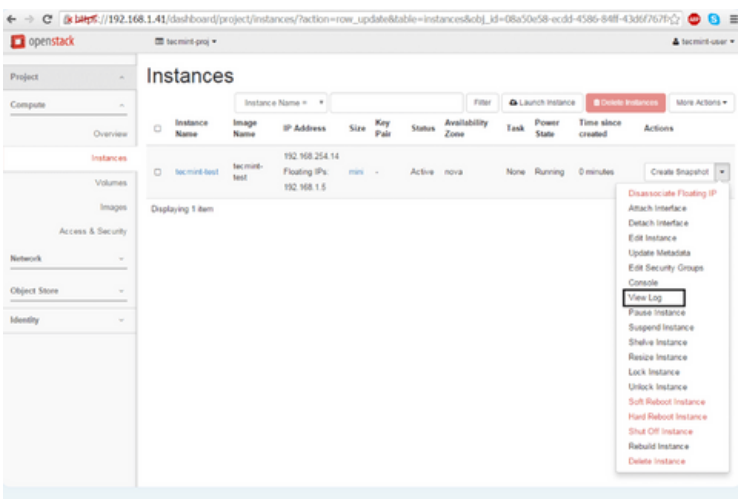
Step 3: Launch an Image Instance in OpenStack

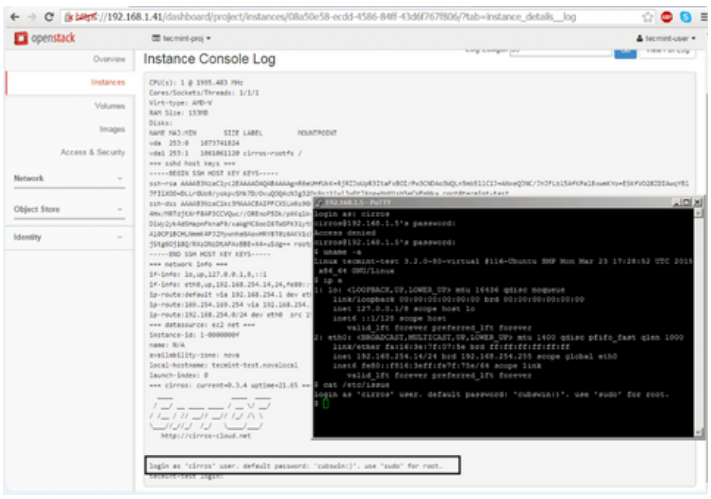




To test the network connectivity for your active virtual machine issue a ping command against the instance floating IP address from a remote computer in your LAN.









Load Balancing and Auto-Scaling

OBJECTIVE:

The goal of this phase is to demonstrate how to enhance service availability and performance by deploying a Load Balancer as a Service (LBaaS) using OpenStack's Octavia, and configuring an auto-scaling policy with Heat, OpenStack's orchestration tool. Together, these tools enable dynamic traffic distribution and automatic resource scaling based on system load.

DEPLOYING A LOAD BALANCER AS A SERVICE (LBAAS)

Overview :

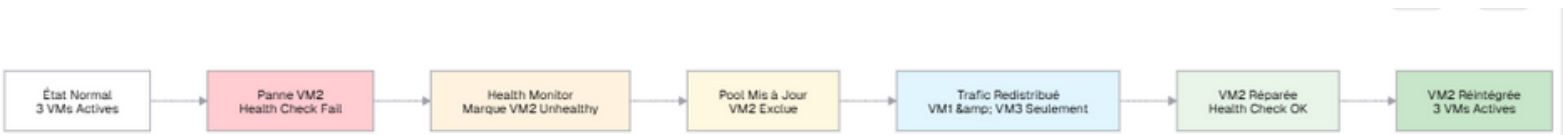
LBaaS allows you to direct incoming network traffic to multiple backend instances (VMs), improving redundancy and balancing system load. In OpenStack, this service is typically managed through Octavia, which automates the creation and configuration of virtual load balancers.

OAD BALANCING WITH OCTAVIA

Load balancing ensures that incoming network traffic is distributed evenly across multiple backend virtual machines (VMs). This avoids overloading any single instance and increases fault tolerance. In OpenStack, this functionality is provided by Octavia, a service dedicated to managing load balancers.

The process begins by creating a virtual load balancer that is assigned a floating IP address for external access. A listener is defined to handle incoming traffic on a specific port (e.g., HTTP on port 80). A pool of backend servers is then associated with the listener. Traffic is routed to these backend servers based on a load balancing algorithm, such as round robin. Health monitors are used to continuously check the status of each backend VM, ensuring that traffic is only sent to healthy instances.

This setup enhances the reliability of web services by allowing the system to continue operating even if one or more instances fail.

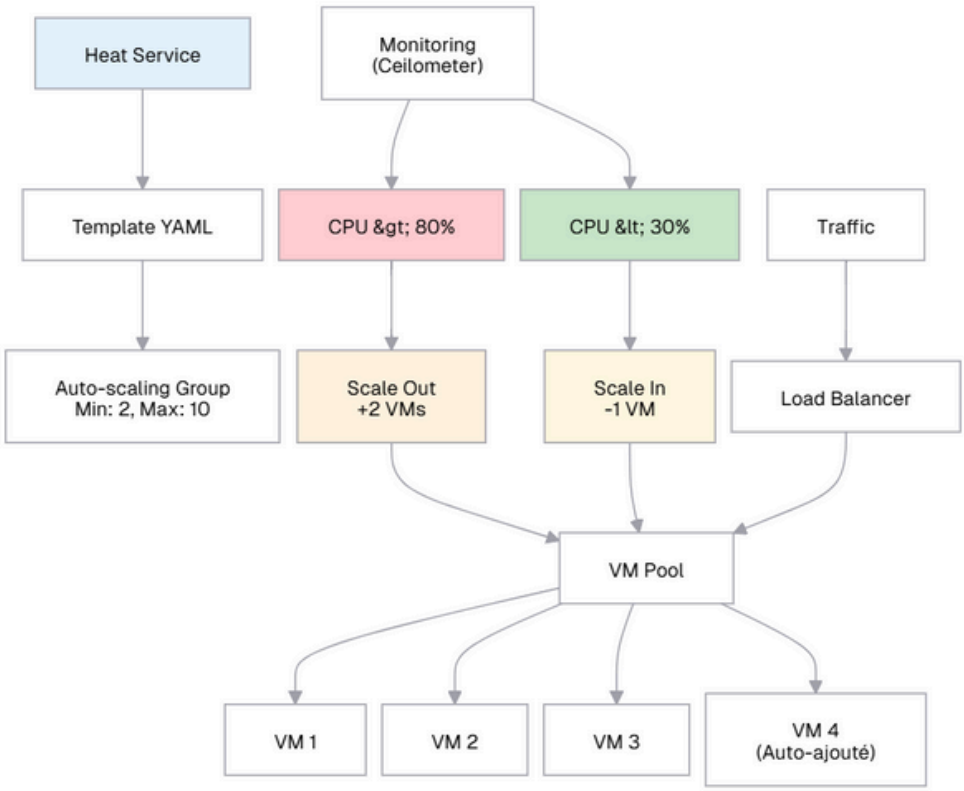


AUTO-SCALING WITH HEAT

Auto-scaling refers to the ability of the cloud platform to automatically increase or decrease the number of active VM instances based on current system load or predefined metrics such as CPU usage. OpenStack achieves this functionality through Heat, its orchestration service.

Heat uses templates written in YAML format to define the desired cloud infrastructure, including the auto-scaling rules. These templates specify a group of identical VMs and outline policies that determine when to scale in (reduce instances) or scale out (add instances). The scaling actions are typically triggered by monitoring tools such as Ceilometer or Gnocchi, which track system metrics and raise alarms when thresholds are exceeded.

When the system load increases beyond a defined threshold, the auto-scaling group adds new VMs to handle the traffic. Once the load decreases, extra instances are terminated automatically, conserving resources and optimizing cost.



BENEFITS AND IMPACT

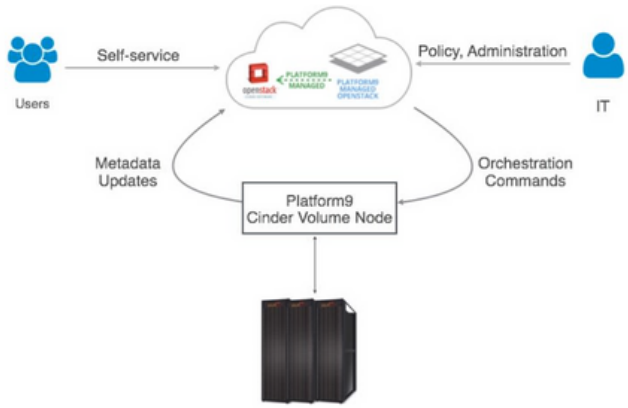
By combining load balancing and auto-scaling, the cloud infrastructure becomes highly resilient and adaptable. Load balancing ensures continuous availability of services, while auto-scaling provides the flexibility to respond to traffic spikes without manual intervention. These capabilities are essential in real-world production environments where performance, uptime, and cost-efficiency are critical.

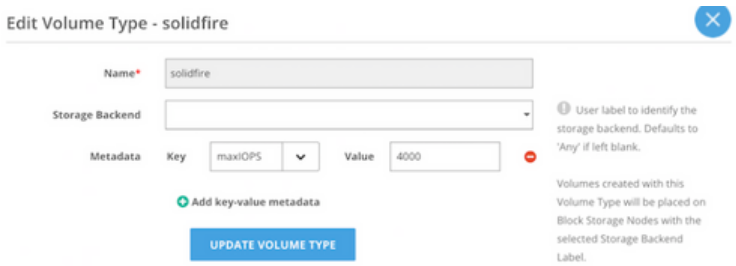
Though these features were optional in this project, they represent a significant step toward building enterprise-grade cloud architectures and should be considered in future implementations or research extensions.

Storage and Networking Setup

CONFIGURING BLOCK STORAGE WITH CINDER

OpenStack's Cinder service delivers persistent block storage that can be attached to virtual machines, just like physical drives. Cinder is structured with a modular and pluggable architecture, supporting integrations with both commodity servers using Linux LVM and enterprise-grade storage arrays such as SolidFire or NetApp.
In smaller deployments, a basic approach involves configuring a Cinder Volume Node using LVM (Logical Volume Manager). This node typically resides on a local server with internal disks, turning it into a simple storage array. In this setup, only the Cinder Volume service runs on-premises, while the rest of the control services (e.g., API, Scheduler) remain centralized on the OpenStack Controller.
Example Setup (LVM-based):
A Dell R720 server using eight 600 GB SAS drives in RAID-10 may yield ~2.3 TB usable Cinder volume capacity. However, scaling beyond a few nodes introduces redundancy and capacity limitations.
To overcome this, enterprise storage integration is preferred. Arrays like SolidFire offer features such as high availability, compression, thin provisioning, deduplication, and Quality of Service (QoS)—all crucial for production environments.
Platform9 simplifies such integrations by allowing storage arrays to be configured directly from its dashboard or CLI, enabling administrators to assign volume types (e.g., SSD-backed, SAS-backed) tailored to specific workloads. These volumes can then be attached to instances and managed seamlessly by users.





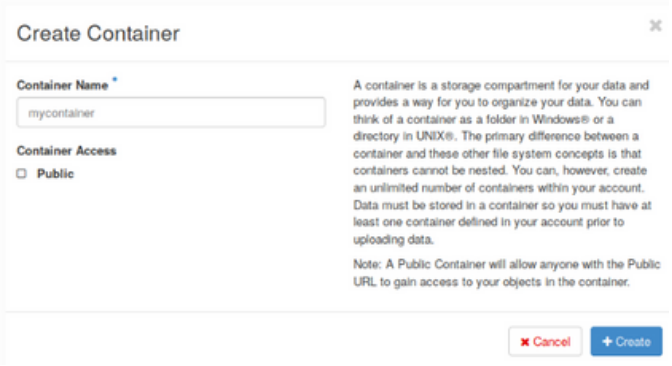
USING OBJECT STORAGE (SWIFT) TO UPLOAD/DOWNLOAD FILES

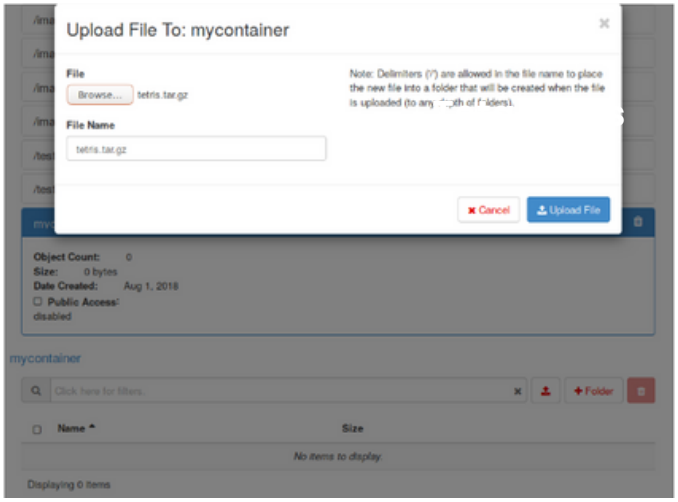
While Cinder focuses on block storage, Swift manages unstructured data through object storage. It's optimal for storing large datasets such as backups, logs, images, and media files. Users interact with Swift via REST APIs or CLI to create containers, upload objects, and download them when needed.
Typical Workflow:openstack container create reports
openstack object create reports backup-2025-06-05.zip
openstack object list reports
openstack object save reports backup-2025-06-05.zip

Storage and Networking Setup

USING OBJECT STORAGE (SWIFT) TO UPLOAD/DOWNLOAD FILES

This model ensures high redundancy and scalability. Swift automatically replicates objects across storage nodes, offering durability without the overhead of filesystem management.





CREATING AND CONFIGURING PRIVATE AND PUBLIC NETWORKS USING NEUTRON

Neutron, OpenStack's networking component, enables cloud administrators to create fully isolated virtual networks and connect them with public infrastructure. The configuration begins by defining private networks for internal VM traffic, and public networks for floating IP access to the internet.

Step-by-step Example:

```
# Private network
openstack network create private-net
openstack subnet create --network private-net --subnet-range 192.168.100.0/24 private-subnet
# Public network
openstack network create --external --provider-network-type flat --provider-physical-network public public-net
openstack subnet create --network public-net --subnet-range 203.0.113.0/24 --no-dhcp --gateway 203.0.113.1 public-subnet
```

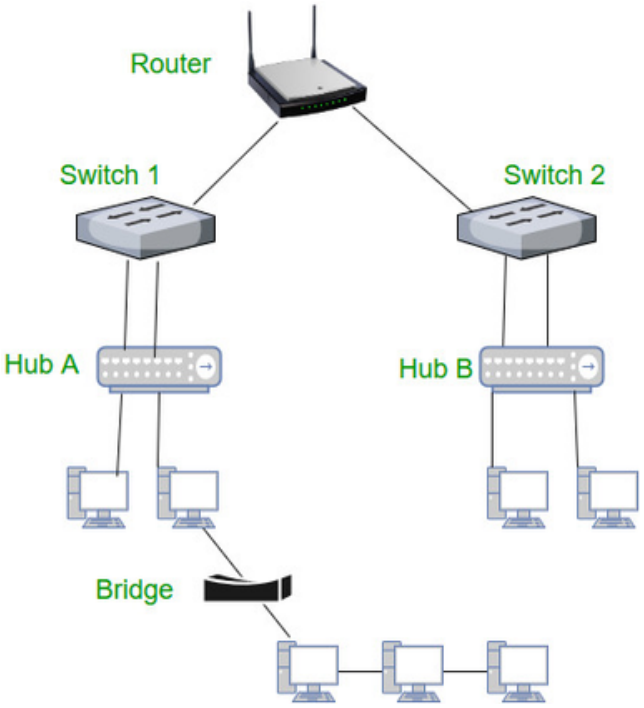
Router setup:

```
openstack router create main-router
openstack router add subnet main-router private-subnet
openstack router set --external-gateway public-net main-router
```

This design mimics real-world cloud environments by isolating tenant traffic while still enabling public exposure when needed.

Storage and Networking Setup

CREATING AND CONFIGURING PRIVATE AND PUBLIC NETWORKS USING NEUTRON



CONNECTING VMS ACROSS NETWORKS

After networks are created, multiple VMs can be launched on different subnets and made to communicate either internally (via private IPs) or externally (using floating IPs). This requires that proper security groups be in place, and routing via Neutron be correctly configured.

Key Steps:

Assign floating IP

openstack floating ip create public-net

openstack server add floating ip web-server 203.0.113.25

Verify connectivity

ping 192.168.100.5 # Internal

curl http://203.0.113.25 # External

In more complex environments, VMs in different private networks can also be connected using shared routers or provider networks, depending on the network isolation policy.

Challenges and Solutions

TECHNICAL DIFFICULTIES DURING SETUP OR DEPLOYMENT

During the course of this project, we encountered several technical challenges, particularly during the installation and deployment phases. One of the first issues we faced was related to package dependencies and compatibility. Some system packages required by DevStack were either missing or outdated, which caused the installation process to fail midway. Additionally, network configuration presented problems, especially when setting the HOST_IP value incorrectly in the local.conf file, leading to failed service bindings and dashboard inaccessibility.

We also experienced intermittent service failures when launching OpenStack components such as Nova and Neutron. These services would sometimes start but then crash due to misconfigured environment variables or insufficient system resources. Furthermore, assigning floating IPs and ensuring external access required careful attention to security group rules, which we initially underestimated.

HOW THEY WERE ADDRESSED

To resolve these issues, we adopted a systematic debugging approach. For dependency errors, we used the sudo apt install -f command and ensured our system was fully updated before each installation attempt. We revalidated all variables in the local.conf file, particularly HOST_IP, by checking the server's actual IP using ip a.

For services that failed to start, we examined logs located in /opt/stack/logs/, which provided useful insights into misconfigurations and missing services. We frequently used the DevStack scripts ./unstack.sh, ./clean.sh, and ./stack.sh to reset the environment and reattempt the installation with corrected settings.

To fix external access issues, we reviewed and adjusted the security group rules, adding explicit permissions for HTTP, HTTPS, and SSH traffic. We also tested the VM connectivity using tools like curl, ping, and browser access to verify that all configurations were correctly applied.

LESSONS LEARNED AND BEST PRACTICES

This project reinforced several key lessons about working with cloud infrastructure:

- Environment consistency is critical: A clean, well-prepared system is essential for smooth installation. Running updates, checking resource availability, and avoiding unnecessary background services can prevent many issues.
- Logs are invaluable: Instead of guessing the cause of a problem, examining detailed logs helped us resolve service failures quickly and accurately.
- Security configuration must not be overlooked: Understanding how security groups and firewall rules function is crucial for ensuring access without compromising the system.
- Iterative testing leads to stability: We learned to apply configurations in small, testable steps—such as assigning a floating IP before launching a web server—allowing for better control and faster recovery from errors.

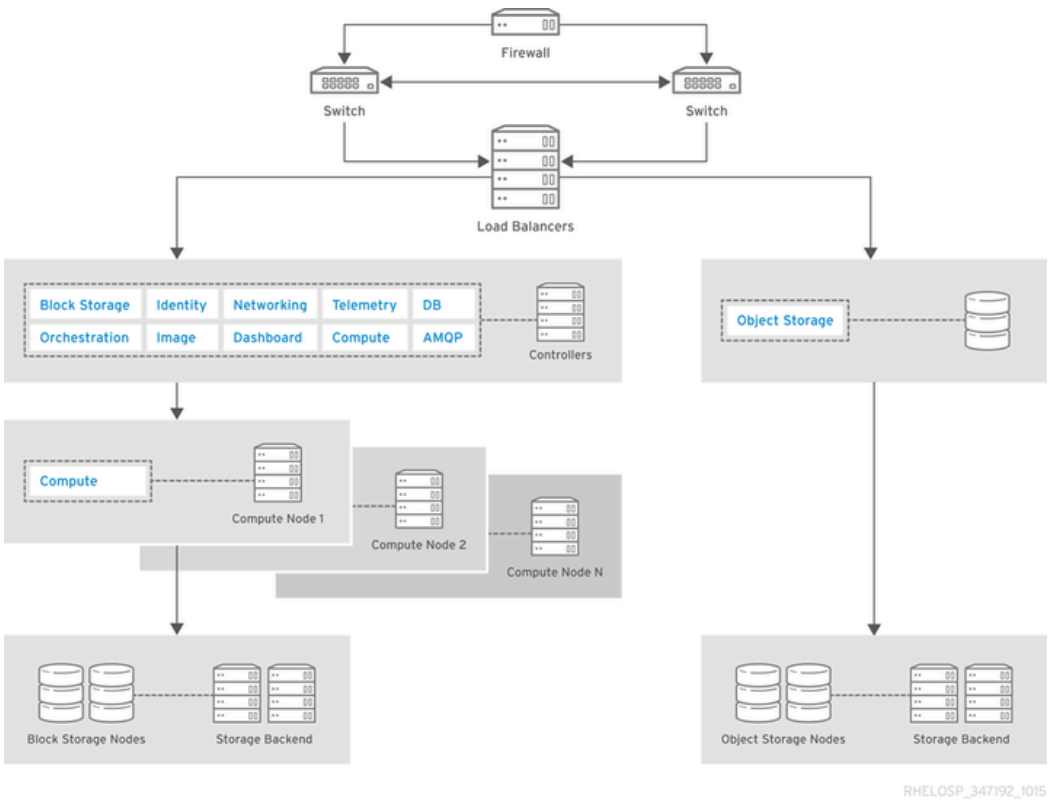
Future Improvements and Real-World Applications

SCALABILITY AND PRODUCTION DEPLOYMENT POTENTIAL

OpenStack is architected for horizontal scalability, allowing organizations to expand their infrastructure by adding more servers rather than upgrading existing ones. This design enables seamless scaling of compute, storage, and networking resources to meet growing demands.

Key Features:

- Horizontal Scaling: Add more nodes (compute, storage, network) to handle increased workloads without significant downtime.
- Modular Architecture: Deploy and manage individual components (e.g., Nova for compute, Neutron for networking) independently, facilitating targeted scaling and maintenance.
- High Availability (HA): Implement redundancy and failover mechanisms to ensure continuous service availability, crucial for production environments.



Future Improvements and Real-World Applications

INTEGRATION WITH OTHER CLOUD TOOLS (CI/CD, MONITORING, ETC.)

Integrating OpenStack with Continuous Integration/Continuous Deployment (CI/CD) pipelines and monitoring tools enhances automation, efficiency, and reliability in cloud operations.

CI/CD Integration:

- Zuul: An open-source CI/CD system designed to automate testing and deployment of changes, ensuring code quality and accelerating development cycles.
- Jenkins: Widely used for automating parts of software development, Jenkins can be integrated with OpenStack to manage infrastructure provisioning and application deployment.

Monitoring Integration:

- Monasca: OpenStack's monitoring-as-a-service solution, providing scalable and performant monitoring capabilities, including metrics collection, alarm generation, and notification.
- Prometheus & Grafana: Popular open-source tools for monitoring and visualization, which can be integrated with OpenStack to provide real-time insights into system performance.

INDUSTRIAL USE CASES FOR OPENSTACK

OpenStack's versatility makes it suitable for various industries, providing tailored solutions to meet specific requirements.

1. Telecommunications:

- Use Case: Telecom companies utilize OpenStack to manage Network Functions Virtualization (NFV), enabling dynamic provisioning of network services.
- Example: China Mobile employs OpenStack to support its 5G network infrastructure, benefiting from its scalability and flexibility.

2. Finance:

- Use Case: Financial institutions leverage OpenStack to build private clouds, ensuring data security, compliance, and rapid deployment of services.
- Example: Walmart adopted OpenStack to enhance its e-commerce platform, achieving improved scalability and cost efficiency.

Future Improvements and Real-World Applications

HIGH AVAILABILITY (HA) AND FAULT TOLERANCE

To move beyond experimental deployments and towards production-grade environments, implementing high availability is essential. In our current setup, most services run on a single node, which poses a single point of failure.

Deploying OpenStack in a multi-node architecture with load balancers, redundant controllers, and replicated databases (e.g., using MariaDB Galera clusters) will ensure fault tolerance and service continuity. Tools like HAProxy and Pacemaker can be integrated to manage failover and distribute traffic evenly.

CONTAINER INTEGRATION AND HYBRID CLOUD COMPATIBILITY

OpenStack is increasingly being used alongside container orchestration platforms like Kubernetes to support microservices architectures.

Future Improvement:

By integrating Magnum, the OpenStack Container Orchestration Engine, we can enable users to provision Kubernetes clusters directly from within OpenStack. This paves the way for hybrid cloud models where workloads can move seamlessly between VM-based and container-based environments.

Conclusion

This project has provided us with a valuable end-to-end experience in deploying and managing a private cloud environment using OpenStack. From understanding the core principles of cloud computing to successfully installing DevStack, creating virtual machines, configuring storage and networking, and deploying web servers, we have gained practical skills that closely mirror real-world industry scenarios. Along the way, we tackled and resolved technical challenges—such as service failures, network misconfigurations, and firewall rules—building a strong foundation in system troubleshooting and cloud orchestration. Beyond technical proficiency, this project enhanced our collaboration, problem-solving, and infrastructure design skills. It also reinforced the importance of automation, documentation, and security in modern IT operations. For future student projects, we recommend focusing not only on building working systems but also on exploring integrations with CI/CD pipelines, monitoring solutions, and high-availability setups, to better reflect enterprise-grade cloud environments. Overall, this hands-on journey with OpenStack has prepared us for more advanced roles in cloud engineering, DevOps, and IT infrastructure management.